BASIC TO INTERMEDIATE:

1 . all customers and the total number of orders they have placed.

```
SELECT
   c.customerid,
   c.companyname,COUNT(o.orderid) AS total_orders
FROM customers c
            LEFT JOIN orders o
   ON c.customerid = o.customerid
GROUP BY
   c.customerid, c.companyname
HAVING COUNT(o.orderid) > 5
ORDER BY
   total_orders DESC;
```

	customerid [PK] character (5)	companyname character varying (40)	total_orders bigint
1	SAVEA	Save-a-lot Markets	31
2	ERNSH	Ernst Handel	30
3	QUICK	QUICK-Stop	28
4	HUNGO	Hungry Owl All-Night Grocers	19
5	FOLKO	Folk och fä HB	19

2. Retrieve the total sales amount per customer by joining customers, orders, and order_details. Show only customers whose total sales exceed 10,000. Sort by total sales descending.

```
SELECT c.customerid ,c.companyname, SUM(od.unitprice * od.quantity * (1-od.discount)) AS sales_amount
FROM customers c
        JOIN orders o ON c.customerid = o.customerid
        JOIN order_details od ON o.orderid = od.orderid
GROUP BY c.customerid ,c.companyname
HAVING SUM(od.unitprice * od.quantity * (1-od.discount)) > 10000
ORDER BY sales_amount DESC;
```

	customerid [PK] character (5)	companyname character varying (40)	sales_amount double precision
1	QUICK	QUICK-Stop	110277.30488350891
2	ERNSH	Ernst Handel	104874.97835350106
3	SAVEA	Save-a-lot Markets	104361.94986812412
4	RATTC	Rattlesnake Canyon Grocery	51097.80048773782
5	HUNGO	Hungry Owl All-Night Groc…	49979.90494634077
6	HANAR	Hanari Carnes	32841.36997813582
7	KOENE	Königlich Essen	30908.383968844264

3 . Get the number of products per category.

Show only categories having more than 10 products. Sort by product count descending.

```
SELECT c.categoryid,c.categoryname, COUNT(p.productid) AS total_products
FROM categories c JOIN products p
        ON c.categoryid = p.categoryid
GROUP BY c.categoryid,c.categoryname
HAVING COUNT(p.productid) > 10
ORDER BY total_products DESC;
```

	categoryid [PK] integer	categoryname character varying (15)	total_products bigint
1	3	Confections	13
2	2	Condiments	12
3	1	Beverages	12
4	8	Seafood	12

4. Display the total quantity sold per product.

Include only products where total quantity sold is greater than 100. Sort by quantity descending.

```
SELECT p.productid,p.productname,SUM(od.quantity) as total_sold
FROM products p JOIN order_details od
        ON p.productid = od.productid
GROUP BY p.productid,p.productname
HAVING SUM(od.quantity) > 100
ORDER BY total_sold DESC;
```

	productid [PK] integer	productname character varying (40)	total_sold bigint
1	60	Camembert Pierrot	1577
2	59	Raclette Courdavault	1496
3	31	Gorgonzola Telino	1397
4	56	Gnocchi di nonna Alice	1263
5	16	Pavlova	1158
6	75	Rhönbräu Klosterbier	1155
7	24	Guaraná Fantástica	1125
8	40	Boston Crab Meat	1103
9	62	Tarte au sucre	1083

5. Find the total number of orders handled by each employee.

Show only employees who handled more than 20 orders. Sort by order count descending.

```
SELECT e.employeeid, CONCAT(e.firstname,' ',e.lastname) as fullname,
COUNT(o.orderid) as total_orders_handled
FROM employees e JOIN orders o
        ON e.employeeid = o.employeeid
GROUP BY e.employeeid
HAVING COUNT(o.orderid) > 20
ORDER BY total_orders_handled DESC;
```

	employeeid [PK] integer	fullname text	total_orders_handled bigint
1	4	Margaret Peaco…	156
2	3	Janet Leverling	127
3	1	Nancy Davolio	123
4	8	Laura Callahan	104
5	2	Andrew Fuller	96
6	7	Robert King	72

INTERMEDIATE

6. Retrieve the total sales per category by joining categories, products, and order_details. Show only categories with total sales above 50,000. Sort by total sales descending.

```
SELECT c.categoryid,c.categoryname,SUM(od.unitprice * od.quantity * (1 - od.discount)) AS total_sales
FROM categories c
        JOIN products p ON c.categoryid = p.categoryid
        JOIN order_details od ON p.productid = od.productid
GROUP BY c.categoryid, c.categoryname
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 50000
ORDER BY total_sales DESC;
```

	categoryid [PK] integer	categoryname character varying (15)	total_sales double precision
1	1	Beverages	267868.1797437187
2	4	Dairy Products	234507.2847417593
3	3	Confections	167357.22484282582
4	6	Meat/Poultry	163022.35939204693
5	8	Seafood	131261.73734469974
6	2	Condiments	106047.08487854415

7. List suppliers and the number of products they supply.

Show only suppliers who supply more than 5 products. Sort by product count descending.

```
SELECT s.supplierid,s.companyname,COUNT(p.productid) AS total_products
FROM suppliers s JOIN products p
        ON s.supplierid = p.supplierid
GROUP BY s.supplierid, s.companyname
HAVING COUNT(p.productid) > 5
ORDER BY total_products DESC;
```

Zero rows because there is no supplier sells more than 5 products

	supplierid [PK] integer	companyname character varying (40)	total_products bigint

8. Get the average unit price per category.

Show only categories where the average price is above 30. Sort by average price descending.

```
SELECT c.categoryid,c.categoryname,AVG(p.unitprice) AS avg_price
FROM categories c JOIN products p
        ON c.categoryid = p.categoryid
GROUP BY c.categoryid, c.categoryname
HAVING AVG(p.unitprice) > 30
ORDER BY avg_price DESC;
```

	categoryid [PK] integer	categoryname character varying (15)	avg_price numeric
1	6	Meat/Poultry	54.0066666666666667
2	1	Beverages	37.9791666666666667
3	7	Produce	32.3700000000000000

9. Display the total revenue generated per employee (orders + order_details).

Show only employees generating more than 20,000 in revenue. Sort by revenue descending.

```
SELECT e.employeeid,CONCAT(e.firstname, ' ', e.lastname) AS fullname,
   SUM(od.unitprice * od.quantity * (1 - od.discount)) AS total_revenue
FROM employees e
        JOIN orders o ON e.employeeid = o.employeeid
        JOIN order_details od ON o.orderid = od.orderid
GROUP BY e.employeeid, e.firstname, e.lastname
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 20000
ORDER BY total_revenue DESC;
```

	employeeid [PK] integer	fullname text	total_revenue double precision
1	4	Margaret Peaco…	232890.84573612877
2	3	Janet Leverling	202812.84285137767
3	1	Nancy Davolio	192107.60437067723
4	2	Andrew Fuller	166537.75483498262
5	8	Laura Callahan	126862.2774064841

10. Retrieve the number of orders shipped to each country.

Show only countries with more than 10 orders. Sort by order count descending.

```
SELECT o.shipcountry,COUNT(o.orderid) AS total_orders_shipped
FROM orders o
GROUP BY o.shipcountry
HAVING COUNT(o.orderid) > 10
ORDER BY total_orders_shipped DESC;
```

	shipcountry character varying (15)	total_orders_shipped bigint
1	USA	122
2	Germany	122
3	Brazil	83
4	France	77
5	UK	56
6	Venezuela	46

ADVANCED:

11. Find customers and the average order value (orders + order_details). Show only customers with average order value greater than 500. Sort by average descending.

```
WITH order_total_details AS (
        SELECT o.customerid, o.orderid,
        SUM(od.unitprice * od.quantity * (1 - od.discount)) AS order_total
        FROM orders o
        JOIN order_details od
        ON o.orderid = od.orderid
        GROUP BY o.orderid, o.customerid
)
SELECT c.customerid, c.companyname,
AVG(otd.order_total) AS avg_order_value
FROM customers c
JOIN order_total_details otd
ON c.customerid = otd.customerid
GROUP BY c.customerid, c.companyname
HAVING AVG(otd.order_total)  > 500
ORDER BY avg_order_value DESC;
```

	customerid [PK] character (5)	companyname character varying (40)	avg_order_value double precision
1	QUICK	QUICK-Stop	3938.4751744110326
2	ERNSH	Ernst Handel	3495.83261178337
3	SAVEA	Save-a-lot Markets	3366.514511874972
4	RATTC	Rattlesnake Canyon Grocery	2838.766693763212
5	HUNGO	Hungry Owl All-Night Grocers	2630.5213129653043
6	SIMOB	Simons bistro	2402.4424992664053

12. Get the top-selling products per category (by total quantity sold).

Show only products with total quantity sold above 200. Sort within category by quantity descending.

```
SELECT c.categoryid,c.categoryname,p.productid,p.productname,
SUM(od.quantity) AS total_sold
FROM categories c
        JOIN products p ON c.categoryid = p.categoryid
        JOIN order_details od ON p.productid = od.productid
GROUP BY c.categoryid, c.categoryname, p.productid, p.productname
HAVING SUM(od.quantity) > 200
ORDER BY c.categoryname, total_sold DESC;
```

categoryid integer	categoryname character varying (15)	productid integer	productname character varying (40)	total_sold bigint
1	Beverages	75	Rhönbräu Klosterbier	1155
1	Beverages	24	Guaraná Fantástica	1125
1	Beverages	2	Chang	1057
1	Beverages	76	Lakkalikööri	981
1	Beverages	35	Steeleye Stout	883
1	Beverages	1	Chai	828

13. Retrieve the total discount given per product (order_details).

Show only products where total discount exceeds 1,000. Sort by discount descending.

```
SELECT p.productid,p.productname,
SUM(od.unitprice * od.quantity * od.discount) AS total_price_discounted
FROM products p JOIN order_details od
        ON p.productid = od.productid
GROUP BY p.productid, p.productname
HAVING SUM(od.unitprice * od.quantity * od.discount) > 1000
ORDER BY total_price_discounted DESC;
```

	productid [PK] integer	productname character varying (40)	total_price_discounted double precision
1	38	Côte de Blaye	8587.465096551552
2	29	Thüringer Rostbratwurst	7367.728025485949
3	59	Raclette Courdavault	5140.300090570003
4	60	Camembert Pierrot	3460.520046980679
5	51	Manjimup Dried Apples	2922.9500460036097
6	18	Carnarvon Tigers	2815.625036600977

14. List employees and the number of unique customers they handled. Show only employees who handled more than 15 unique customers. Sort by count descending.

```
SELECT e.employeeid,CONCAT(e.firstname, ' ', e.lastname) AS fullname,
COUNT(DISTINCT o.customerid) AS unique_customer_count
FROM employees e JOIN orders o
    ON e.employeeid = o.employeeid
GROUP BY e.employeeid, e.firstname, e.lastname
HAVING COUNT(DISTINCT o.customerid) > 15
ORDER BY unique_customer_count DESC;
```

	employeeid [PK] integer	fullname text	unique_customer_count bigint
1	4	Margaret Peaco...	75
2	1	Nancy Davolio	65
3	3	Janet Leverling	63
4	2	Andrew Fuller	59
5	8	Laura Callahan	56

15. Find the monthly total sales (year + month) using orders and order_details. Show only months where total sales exceed 30,000. Sort by year and month ascending.

```
SELECT EXTRACT(YEAR FROM o.orderdate) AS year,
EXTRACT(MONTH FROM o.orderdate) AS month,
SUM(od.unitprice * od.quantity * (1 - od.discount)) AS monthly_sales
FROM orders o
JOIN order_details od ON o.orderid = od.orderid
GROUP BY year, month
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 30000
ORDER BY year, month;
```

	year numeric	month numeric	monthly_sales double precision
1	1996	10	37515.72491448447
2	1996	11	45600.04494312592
3	1996	12	45239.629965431996
4	1997	1	61258.0699304849
5	1997	2	38483.63494294286
6	1997	3	38547.21996988951
7	1997	4	53032.95242693588

STORED PROCEDURES:

1 . Create a stored procedure to insert a new customer. Use a transaction so that if any required value is missing, the insert is rolled back.

```
CREATE OR REPLACE PROCEDURE create_customer(
   p_customerid VARCHAR,
   p_companyname VARCHAR,
   p_contactname VARCHAR,
   p_contacttitle VARCHAR,
   p_address VARCHAR,
   p_city VARCHAR,
   p_region VARCHAR,
   p_postalcode VARCHAR,
   p_country VARCHAR,
   p_phone VARCHAR,
   p_fax VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
   -- Check for missing fields
   IF p_customerid IS NULL OR
      p _companyname IS NULL OR
      p_contactname IS NULL OR
      p_country IS NULL OR
      p_contacttitle IS NULL OR
      p_phone IS NULL THEN


      RAISE NOTICE 'Required field missing. Rolling back';
      ROLLBACK;
      RETURN;
```

```
END IF;

-- Insert if all required fields present
INSERT INTO customers (
   customerid,
   companyname,
   contactname,
   contacttitle,
   address,
   city,
   region,
   postalcode,
   country,
   phone,
   fax
)
VALUES (
   p_customerid,
   p_companyname,
   p_contactname,
   p_contacttitle,
   p_address,
   p_city,
   p_region,
   p_postalcode,
   p_country,
   p_phone,
   p_fax
);

-- commit if insert done
```

```
    COMMIT;
    RAISE NOTICE 'Customer inserted successfully';
END;
$$;

CALL create_customer(
     'C004',
    'Techno Solutions Ltd',
    'Arvind Kumar',
    'Manager',
    '123 Anna Salai',
    'Chennai',
    'Tamil Nadu',
    '600002',
    'India',
    '044-12345678',
    '044-87654321'
);

SELECT * FROM customers WHERE customerid = 'C004';
```

Customer data inserted:

	customerid [PK] character (5)	companyname character varying (40)	contactname character varying (30)	contacttitle character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)
1	C004	Techno Solutions Ltd	Arvind Kumar	Manager	123 Anna Salai	Chennai	Tamil Nadu

IF field Missing:

Data Output | Messages | Notifications

```
NOTICE:  Required field missing. Rolling back transaction.
CALL

Query returned successfully in 47 msec.
```

2. Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

```
CREATE OR REPLACE PROCEDURE place_order(
    p_customerid VARCHAR,
    p_employeeid INT,
    p_productid INT,
    p_quantity INT,
    p_unitprice NUMERIC,
    p_shipaddress VARCHAR,
    p_shipcity VARCHAR,
    p_shipcountry VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_orderid INT;
    v_stock INT;
BEGIN
    -- handle null values
    IF p_customerid IS NULL OR p_employeeid IS NULL OR
       p_productid IS NULL OR p_quantity IS NULL OR
       p_unitprice IS NULL OR p_shipaddress IS NULL OR
       p_shipcity IS NULL OR p_shipcountry IS NULL OR
       p_quantity <= 0 OR p_unitprice <= 0 THEN

        RAISE NOTICE 'Invalid input values';
        ROLLBACK; RETURN;
    END IF;

    -- check for existence
    IF NOT EXISTS (SELECT 1 FROM customers WHERE customerid = p_customerid) THEN
        RAISE NOTICE 'Customer not found';
```

```
    ROLLBACK; RETURN;
END IF;

IF NOT EXISTS (SELECT 1 FROM employees WHERE employeeid = p_employeeid) THEN
    RAISE NOTICE 'Employee not found';
    ROLLBACK; RETURN;
END IF;

-- check for product and stock
SELECT unitsinstock INTO v_stock
FROM products
WHERE productid = p_productid;

IF v_stock IS NULL THEN
    RAISE NOTICE 'Product not found';
    ROLLBACK; RETURN;
END IF;

IF v_stock < p_quantity THEN
    RAISE NOTICE 'Insufficient stock';
    ROLLBACK; RETURN;
END IF;

-- insert
INSERT INTO orders(
    customerid, employeeid, orderdate,
    shipaddress, shipcity, shipcountry
)
VALUES (
    p_customerid, p_employeeid, CURRENT_DATE,
    p_shipaddress, p_shipcity, p_shipcountry
)
```

```
    RETURNING orderid INTO v_orderid;

    -- insert in order details
    INSERT INTO order_details(
        orderid, productid, unitprice, quantity, discount
    )
    VALUES (
        v_orderid, p_productid, p_unitprice, p_quantity, 0
    );

    -- update
    UPDATE products
    SET unitsinstock = unitsinstock - p_quantity
    WHERE productid = p_productid;

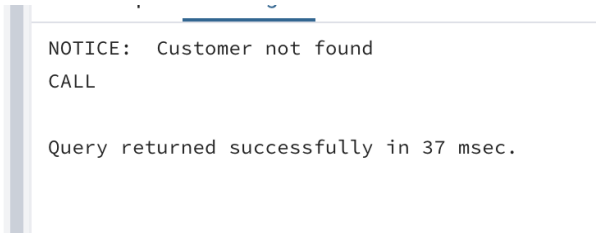
    COMMIT;
    RAISE NOTICE 'Order placed successfully. Order ID: %', v_orderid;

END;
$$;
```

```
NOTICE:  Order placed successfully. Order ID: 11082
CALL

Query returned successfully in 39 msec.
```

IF error :



3. Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

```
CREATE OR REPLACE PROCEDURE update_product_stock (
  p_productid INT,
  p_quantity INT
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_stock INT;
  v_exists INT;
BEGIN
  -- validate input
  IF p_productid IS NULL OR p_quantity IS NULL OR p_quantity <= 0 THEN
    RAISE NOTICE 'Invalid input';
    ROLLBACK; RETURN;
  END IF;

  -- check product
  SELECT COUNT(*) INTO v_exists
  FROM products
  WHERE productid = p_productid;
```

```
  IF v_exists = 0 THEN
    RAISE NOTICE 'Product not found';
    ROLLBACK; RETURN;
  END IF;

  -- get stock
  SELECT unitsinstock INTO v_stock
  FROM products
  WHERE productid = p_productid;

  IF v_stock < p_quantity THEN
    RAISE NOTICE 'Insufficient stock';
    ROLLBACK; RETURN;
  END IF;

  -- update
  UPDATE products
  SET unitsinstock = unitsinstock - p_quantity
  WHERE productid = p_productid;

  COMMIT;
  RAISE NOTICE 'Stock updated successfully';

END;
$$;
CALL update_product_stock(10, 5);
```

```
NOTICE:  Stock updated successfully
CALL

Query returned successfully in 45 msec.
```

If error:

```
NOTICE:  Product not found. Rolling back.
CALL

Query returned successfully in 167 msec.
```

4. Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.

```
CREATE OR REPLACE PROCEDURE cancel_order(
    p_orderid INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_exists INT;
    v_detail_count INT;
BEGIN
    -- validate
    IF p_orderid IS NULL THEN
        RAISE NOTICE 'Order ID cannot be NULL';
        ROLLBACK; RETURN;
    END IF;

    -- check order
    SELECT COUNT(*) INTO v_exists
    FROM orders
    WHERE orderid = p_orderid;

    IF v_exists = 0 THEN
```

```
        RAISE NOTICE 'Order not found';
        ROLLBACK; RETURN;
    END IF;

    -- check details
    SELECT COUNT(*) INTO v_detail_count
    FROM order_details
    WHERE orderid = p_orderid;

    IF v_detail_count = 0 THEN
        RAISE NOTICE 'No order details found';
        ROLLBACK; RETURN;
    END IF;

    -- delete child then parent
    DELETE FROM order_details WHERE orderid = p_orderid;
    DELETE FROM orders WHERE orderid = p_orderid;

    COMMIT;
    RAISE NOTICE 'Order % cancelled successfully', p_orderid;

END;
$$;

CALL cancel_order(10248);
```

```
NOTICE:  Order 10248 cancelled successfully
CALL

Query returned successfully in 35 msec.
```

If invalid:

```
NOTICE:  Order not found
CALL

Query returned successfully in 72 msec.
```

5.Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

```
CREATE OR REPLACE PROCEDURE transfer_supplier_products(
  p_old_supplierid INT,
  p_new_supplierid INT
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_old_exists INT;
  v_new_exists INT;
  v_count INT;
BEGIN
  -- validate input
  IF p_old_supplierid IS NULL OR p_new_supplierid IS NULL
    OR p_old_supplierid = p_new_supplierid THEN
     RAISE NOTICE 'Invalid supplier IDs';
     ROLLBACK; RETURN;
  END IF;

  -- check old supplier
```

```
SELECT COUNT(*) INTO v_old_exists
FROM suppliers
WHERE supplierid = p_old_supplierid;

IF v_old_exists = 0 THEN
   RAISE NOTICE 'Old supplier not found';
   ROLLBACK; RETURN;
END IF;

-- check new supplier
SELECT COUNT(*) INTO v_new_exists
FROM suppliers
WHERE supplierid = p_new_supplierid;

IF v_new_exists = 0 THEN
   RAISE NOTICE 'New supplier not found';
   ROLLBACK; RETURN;
END IF;

-- check products exist for old supplier
SELECT COUNT(*) INTO v_count
FROM products
WHERE supplierid = p_old_supplierid;

IF v_count = 0 THEN
   RAISE NOTICE 'No products to transfer';
   ROLLBACK; RETURN;
END IF;

-- transfer products
UPDATE products
```

```
    SET supplierid = p_new_supplierid
    WHERE supplierid = p_old_supplierid;

    COMMIT;
    RAISE NOTICE 'Products transferred successfully';

END;
$$;

CALL transfer_supplier_products(1, 2);
```

```
NOTICE:  Products transferred successfully
CALL

Query returned successfully in 51 msec.
```

If error:

```
NOTICE:  Old supplier not found
CALL

Query returned successfully in 40 msec.
```

6.Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

```
CREATE OR REPLACE PROCEDURE update_category_price(
   p_categoryid INT,
   p_percentage NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
   v_exists INT;
BEGIN
   -- validate input
   IF p_categoryid IS NULL OR p_percentage IS NULL OR p_percentage <= 0 THEN
      RAISE NOTICE 'Invalid input or percentage must be > 0';
      ROLLBACK; RETURN;
   END IF;

   -- check category
   SELECT COUNT(*) INTO v_exists
   FROM categories
   WHERE categoryid = p_categoryid;

   IF v_exists = 0 THEN
      RAISE NOTICE 'Category not found';
      ROLLBACK; RETURN;
   END IF;

   -- update prices
   UPDATE products
   SET unitprice = unitprice + (unitprice * p_percentage / 100)
```

```
    WHERE categoryid = p_categoryid;

    COMMIT;
    RAISE NOTICE 'Prices updated successfully';

END;
$$;

CALL update_category_price(1, 10);
```

```
NOTICE:  Prices updated successfully
CALL

Query returned successfully in 26 msec.
```

If error:

```
NOTICE:  Category not found
CALL

Query returned successfully in 35 msec.
```

7. Create a stored procedure to add a new product under an existing category and supplier. Rollback if the category or supplier does not exist.

```
CREATE OR REPLACE PROCEDURE add_product(
    p_productname VARCHAR,
    p_supplierid INT,
    p_categoryid INT,
    p_unitprice NUMERIC,
    p_unitsinstock INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cat_exists INT;
    v_sup_exists INT;
BEGIN
    -- validate input
    IF p_productname IS NULL OR p_supplierid IS NULL OR
       p_categoryid IS NULL OR p_unitprice IS NULL OR
       p_unitsinstock IS NULL OR p_unitprice <= 0 OR p_unitsinstock < 0 THEN
        RAISE NOTICE 'Invalid input values';
        ROLLBACK; RETURN;
    END IF;

    -- check category
    SELECT COUNT(*) INTO v_cat_exists
    FROM categories
    WHERE categoryid = p_categoryid;

    IF v_cat_exists = 0 THEN
        RAISE NOTICE 'Category not found';
        ROLLBACK; RETURN;
```

```
    END IF;

    -- check supplier
    SELECT COUNT(*) INTO v_sup_exists
    FROM suppliers
    WHERE supplierid = p_supplierid;

    IF v_sup_exists = 0 THEN
        RAISE NOTICE 'Supplier not found';
        ROLLBACK; RETURN;
    END IF;

    -- insert product
    INSERT INTO products(
        productname, supplierid, categoryid, unitprice, unitsinstock
    )
    VALUES (
        p_productname, p_supplierid, p_categoryid, p_unitprice, p_unitsinstock
    );

    COMMIT;
    RAISE NOTICE 'Product added successfully';

END;
$$;
```

```
NOTICE:  Product added successfully
CALL

Query returned successfully in 39 msec.
```

8.Create a stored procedure to delete a customer only if the customer has no orders. Use a transaction and rollback if orders exist.

```
CREATE OR REPLACE PROCEDURE delete_customer_if_no_orders(
    p_customerid VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_exists INT;
    v_order_count INT;
BEGIN
    -- validate input
    IF p_customerid IS NULL THEN
        RAISE NOTICE 'Customer ID cannot be NULL';
        ROLLBACK; RETURN;
    END IF;

    -- check customer
    SELECT COUNT(*) INTO v_exists
    FROM customers
    WHERE customerid = p_customerid;

    IF v_exists = 0 THEN
        RAISE NOTICE 'Customer not found';
        ROLLBACK; RETURN;
    END IF;

    -- check orders
    SELECT COUNT(*) INTO v_order_count
    FROM orders
    WHERE customerid = p_customerid;
```

```
    IF v_order_count > 0 THEN
        RAISE NOTICE 'Cannot delete: customer has orders';
        ROLLBACK; RETURN;
    END IF;

    -- delete customer
    DELETE FROM customers
    WHERE customerid = p_customerid;

    COMMIT;
    RAISE NOTICE 'Customer deleted successfully';

END;
$$;
select * from customers;
CALL delete_customer_if_no_orders('C004');
```

```
NOTICE:  Customer deleted successfully
CALL

Query returned successfully in 50 msec.
```

If error:

```
NOTICE:  Cannot delete: customer has orders
CALL

Query returned successfully in 33 msec.
```

9.Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

```
CREATE OR REPLACE PROCEDURE apply_discount_to_order(
    p_orderid INT,
    p_discount NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_exists INT;
BEGIN
    -- validate input
    IF p_orderid IS NULL OR p_discount IS NULL OR p_discount < 0 THEN
        RAISE NOTICE 'Invalid input';
        ROLLBACK; RETURN;
    END IF;

    -- check order
    SELECT COUNT(*) INTO v_exists
    FROM orders
    WHERE orderid = p_orderid;

    IF v_exists = 0 THEN
        RAISE NOTICE 'Order not found';
        ROLLBACK; RETURN;
    END IF;

    -- apply discount
    UPDATE order_details
    SET discount = p_discount
```

```
    WHERE orderid = p_orderid;

    COMMIT;
    RAISE NOTICE 'Discount applied successfully';

END;
$$;
CALL apply_discount_to_order(10248, 0.10);
```

```
NOTICE:  Order not found
CALL

Query returned successfully in 87 msec.
```

10.Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

```
CREATE OR REPLACE PROCEDURE place_order_multiple_products(
    p_customerid VARCHAR,
    p_employeeid INT,
    p_items JSON   -- [{"productid":1,"qty":2,"price":10}, ...]
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_orderid INT;
    v_prod_id INT;
    v_qty INT;
    v_price NUMERIC;
```

```
    v_stock INT;
    v_exists INT;
    item JSON;
BEGIN
    -- validate input
    IF p_customerid IS NULL OR p_employeeid IS NULL OR p_items IS NULL THEN
        RAISE NOTICE 'Invalid input';
        ROLLBACK; RETURN;
    END IF;

    -- check customer
    SELECT COUNT(*) INTO v_exists FROM customers WHERE customerid = p_customerid;
    IF v_exists = 0 THEN
        RAISE NOTICE 'Customer not found';
        ROLLBACK; RETURN;
    END IF;

    -- check employee
    SELECT COUNT(*) INTO v_exists FROM employees WHERE employeeid = p_employeeid;
    IF v_exists = 0 THEN
        RAISE NOTICE 'Employee not found';
        ROLLBACK; RETURN;
    END IF;

    -- create order
    INSERT INTO orders(customerid, employeeid, orderdate)
    VALUES (p_customerid, p_employeeid, CURRENT_DATE)
    RETURNING orderid INTO v_orderid;

    -- loop items
    FOR item IN SELECT * FROM json_array_elements(p_items)
```

```
LOOP
  v_prod_id := (item->>'productid')::INT;
  v_qty    := (item->>'qty')::INT;
  v_price   := (item->>'price')::NUMERIC;

  -- validate item
  IF v_prod_id IS NULL OR v_qty IS NULL OR v_price IS NULL OR v_qty <= 0 THEN
    RAISE NOTICE 'Invalid item data';
    ROLLBACK; RETURN;
  END IF;

  -- check product + stock
  SELECT unitsinstock INTO v_stock
  FROM products
  WHERE productid = v_prod_id;

  IF v_stock IS NULL THEN
    RAISE NOTICE 'Product % not found', v_prod_id;
    ROLLBACK; RETURN;
  END IF;

  IF v_stock < v_qty THEN
    RAISE NOTICE 'Insufficient stock for product %', v_prod_id;
    ROLLBACK; RETURN;
  END IF;

  -- insert order item
  INSERT INTO order_details(orderid, productid, unitprice, quantity, discount)
  VALUES (v_orderid, v_prod_id, v_price, v_qty, 0);

  -- update stock
```

```
    UPDATE products
    SET unitsinstock = unitsinstock - v_qty
    WHERE productid = v_prod_id;
  END LOOP;

  COMMIT;
  RAISE NOTICE 'Order placed successfully. Order ID: %', v_orderid;

END;
$$;

CALL place_order_multiple_products(
  'ALFKI',
  1,
  '[{"productid":1,"qty":2,"price":10},
   {"productid":2,"qty":3,"price":20}]'
);
```

```
NOTICE:  Order placed successfully. Order ID: 11084
CALL

Query returned successfully in 41 msec.
```